МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «МКиИТ»

Лабораторная работа № 4
по дисциплине «Введение в информационные технологии»
по теме: «Организация Python-проекта»

Выполнил студент группы:
БВТ2604
Яшкольдина Анастасия Дмитриевна
Проверил: Павиликов Артем Евгеньевич

Москва 2026

Оглавление

Введение.....3
Выполнение.....4
Выводы.....26

Цель и задачи работы:

Целью данной лабораторной работы является: изучение принципов организации Python-проекта: разбиения программы на модули и пакеты, использования точки входа, работы с файлами и путями, обработки ошибок, логирования, конфигурационных файлов, виртуального окружения и системы контроля версий Git.

Задачи: создать структуру проекта измерений; разделить код на модули пакета utils; организовать точку входа if __name__ == "__main__"; реализовать чтение данных из файла с помощью pathlib; обработать ошибки (try/except/else/finally, raise); настроить логирование в файл; вынести настройки в config.json; создать виртуальное окружение, requirements.txt и .gitignore; зафиксировать выполнение каждого задания отдельным коммитом.

Выполнение

1. Структура проекта

Создан проект measurement_project со структурой: каталоги data/ (входные данные), logs/ (журнал работы), utils/ (пакет с модулями reader.py и statistics.py), файлы main.py, config.json и .gitignore. Наличие файла __init__.py в utils/ делает каталог пакетом Python. Структура выведена командой tree /F.

```
# Задание 1
# структура проекта (tree /F)
measurement_project/
├── data/
├── logs/
├── utils/
│   ├── __init__.py
│   ├── reader.py
│   └── statistics.py
├── config.json
├── .gitignore
└── main.py
```

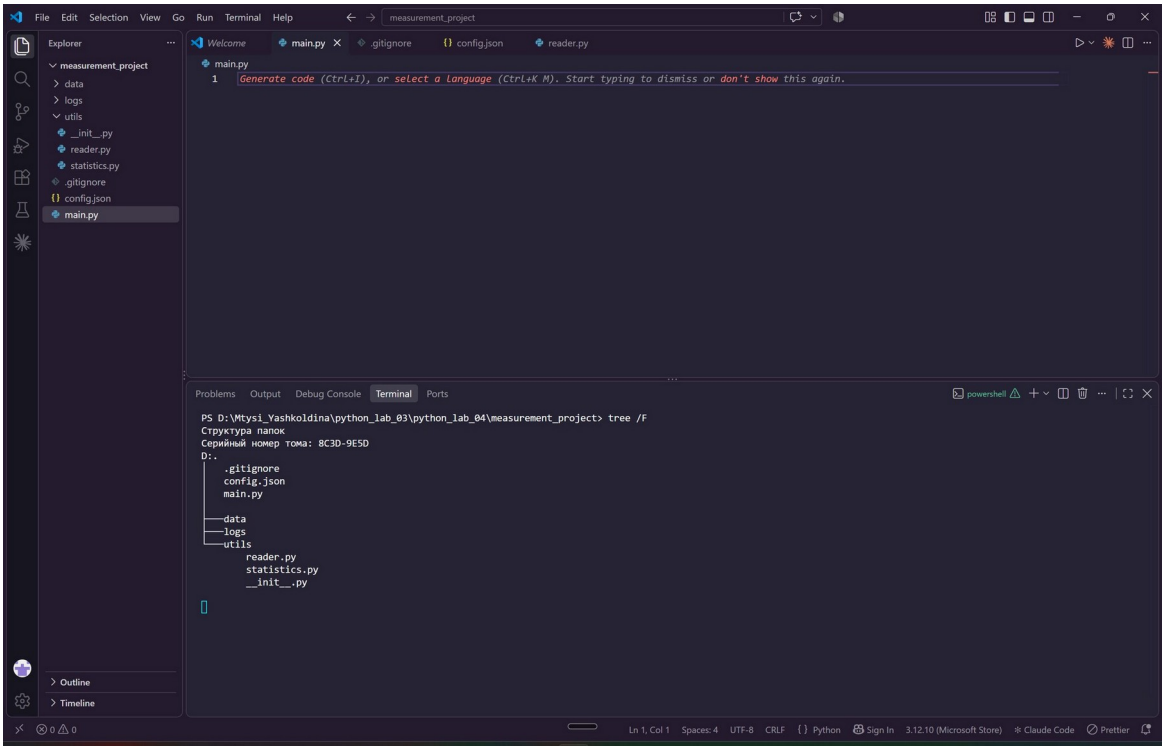


Рисунок 1. – Результат выполнения задания 1.

2. Модули и импорт

В модуле utils/statistics.py реализованы функции calculate_average(), find_min() и find_max(). В main.py они импортируются командой from

utils.statistics import ... и применяются к тестовому списку измерений (среднее значение 12.88).

```
# Задание 2
# файл utils/statistics.py
def calculate_average(numbers):
    return sum(numbers) / len(numbers)


def find_min(numbers):
    m = numbers[0]
    for n in numbers:
        if n < m:
            m = n
    return m


def find_max(numbers):
    m = numbers[0]
    for n in numbers:
        if n > m:
            m = n
    return m

# файл main.py
from utils.statistics import calculate_average, find_min,
find_max

numbers = [12.5, 13.0, 11.8, 14.2, 12.9]

print("Мин:", find_min(numbers), "Макс:", find_max(numbers),
"Среднее:", calculate_average(numbers))
```

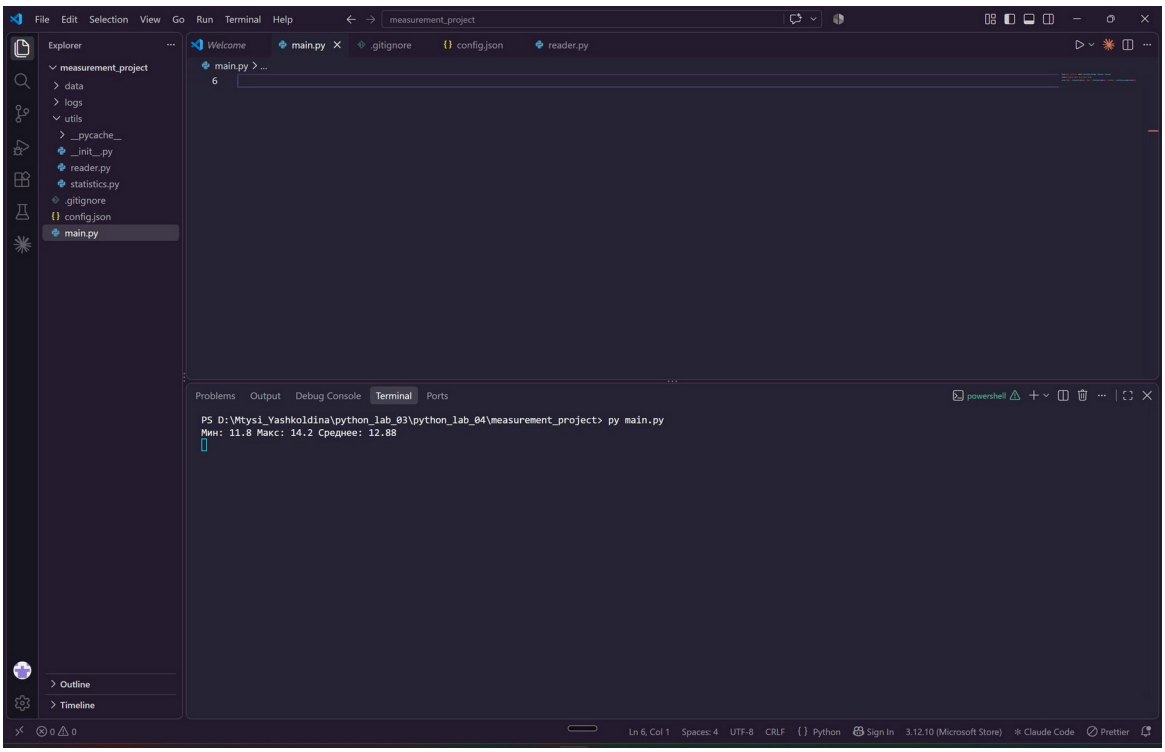


Рисунок 2. – Результат выполнения задания 2.

3. Точка входа и __name__

Код main.py помещён в функцию main(), которая вызывается только при условии if __name__ == "__main__". Во вспомогательном файле test_import.py выполнен import main: при импорте переменная __name__ равна "main", а не "__main__", поэтому main() не запускается, что и видно по выводу. После проверки test_import.py удалён.

```
# Задание 3
# файл main.py
from utils.statistics import calculate_average, find_min,
find_max


def main():
    numbers = [12.5, 13.0, 11.8, 14.2, 12.9]
    print("Мин:", find_min(numbers), "Макс:", find_max(numbers),
"Среднее:", calculate_average(numbers))


if __name__ == "__main__":
    main()

# файл test_import.py
import main

print("main.py импортирован, main() не запускался")
```

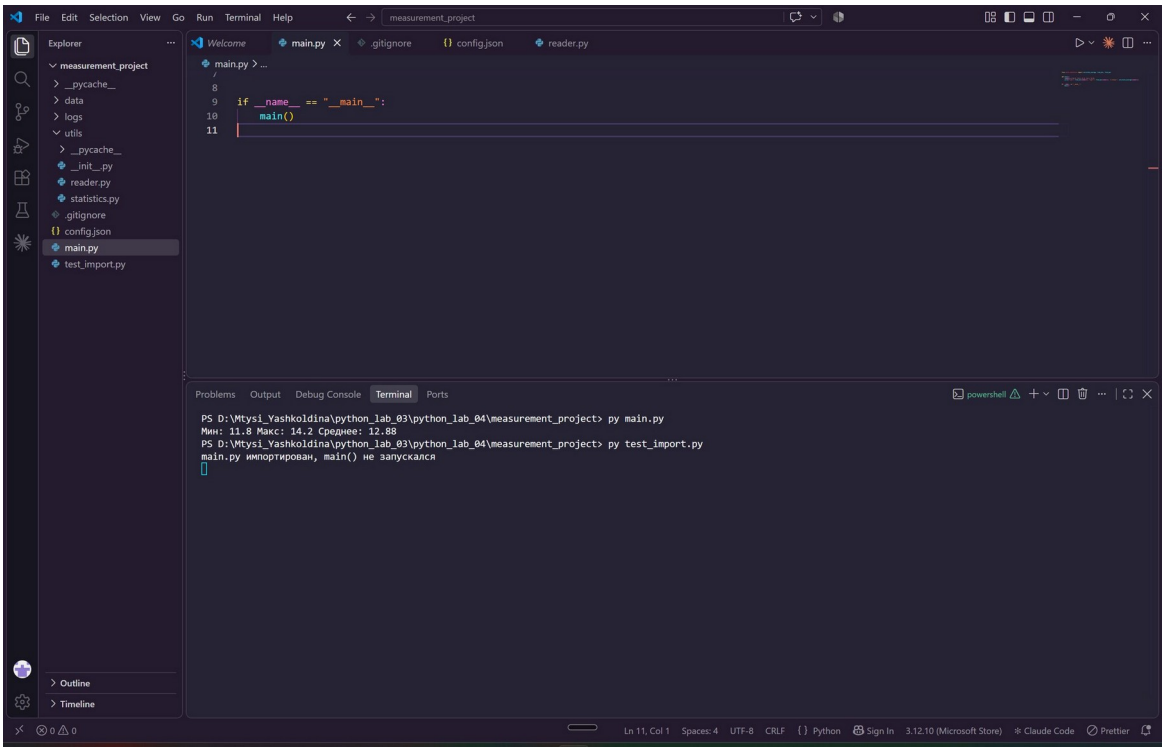


Рисунок 3. – Результат выполнения задания 3.

4. Чтение данных из файла

В модуле utils/reader.py создана функция read_numbers(file_path), читающая числа из файла построчно (пустые строки пропускаются) и возвращающая список. Данные хранятся в data/measurements.txt, программа выводит количество значений, минимум, максимум и среднее.

```
# Задание 4
# файл utils/reader.py
def read_numbers(file_path):
    numbers = []
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            line = line.strip()
            if line:
                numbers.append(float(line))
    return numbers

# файл main.py
from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    numbers = read_numbers("data/measurements.txt")
    print("Значений:", len(numbers), "Мин:", find_min(numbers),
"Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))


if __name__ == "__main__":
    main()
```

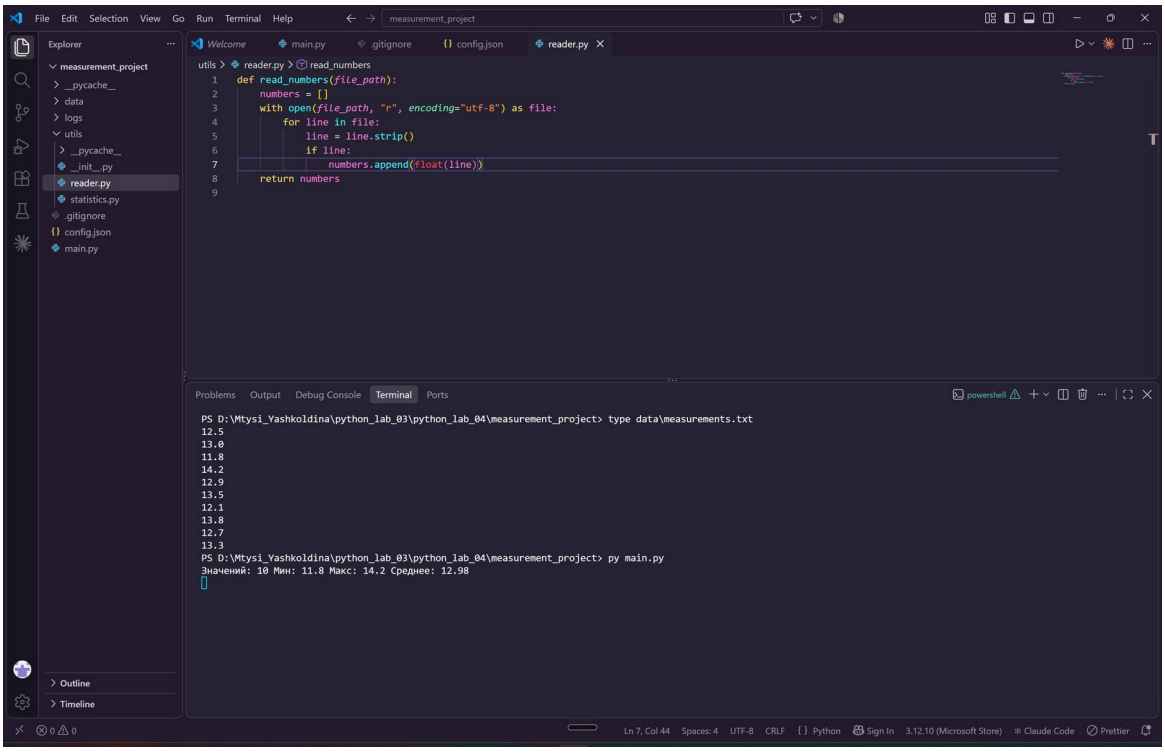


Рисунок 4. – Результат выполнения задания 4.

5. Работа с путями (pathlib)

Путь к файлу формируется через pathlib.Path, программа выводит текущий каталог (Path.cwd()), проверяет существование файла (path.exists()) и создаёт каталог logs/ методом mkdir(exist_ok=True).

```
# Задание 5
# файл main.py
from pathlib import Path

from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)
    print("Текущий каталог:", Path.cwd())
    print("Файл существует:", path.exists())
    numbers = read_numbers(path)
    print("Значений:", len(numbers), "Мин:", find_min(numbers),
"Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))


if __name__ == "__main__":
    main()
```

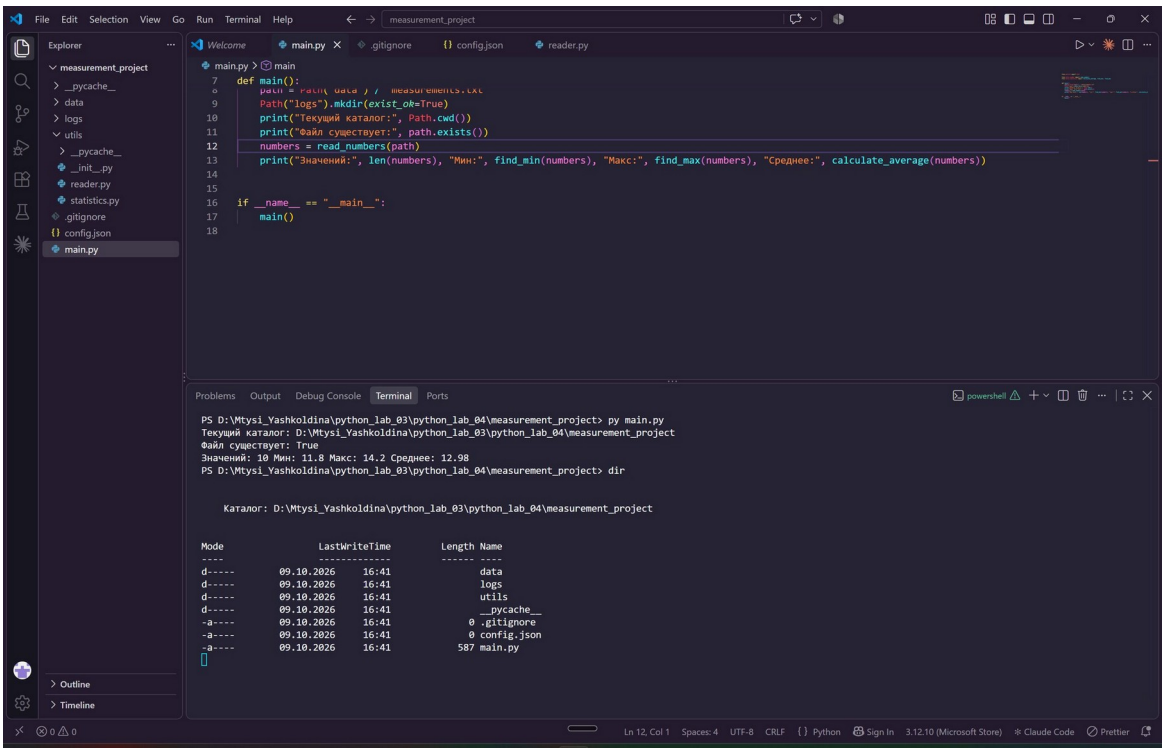


Рисунок 5. – Результат выполнения задания 5.

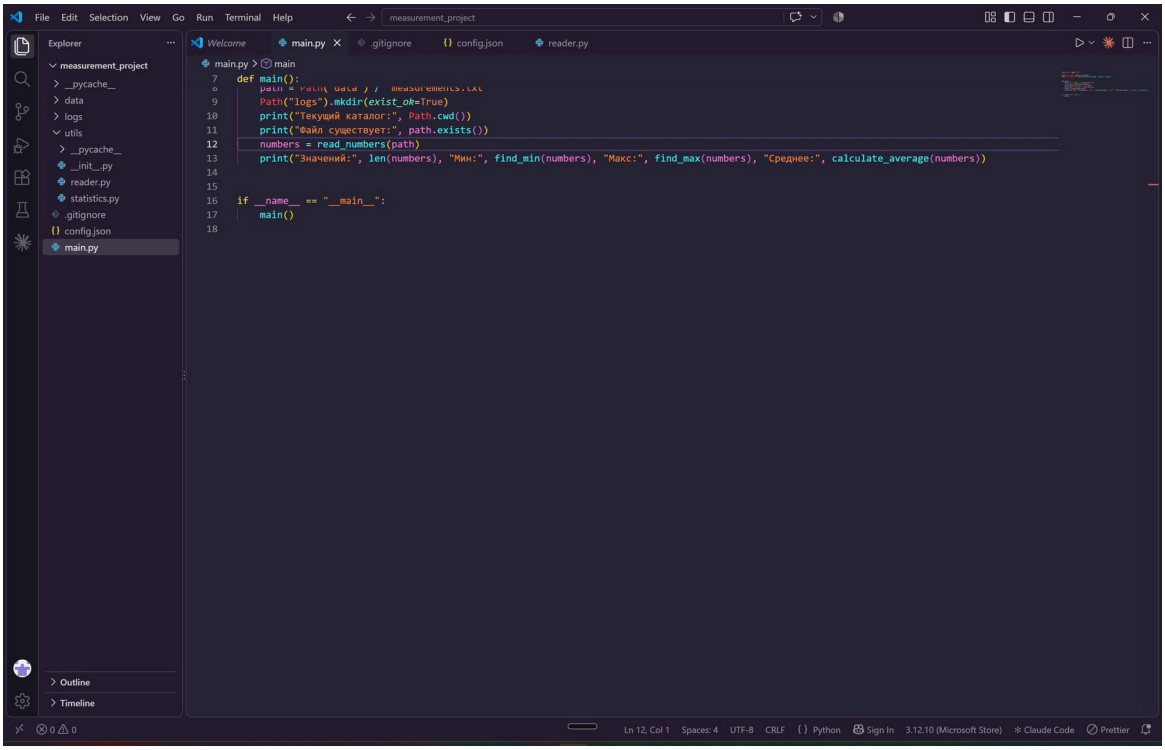


Рисунок 6. – Код main.py задания 5.

6. Обработка исключений

Чтение файла обёрнуто в try/except/else: ошибка FileNotFoundError (файл отсутствует) и ValueError (в файле некорректное значение) обрабатываются с понятным сообщением, а блок else выполняется только при успешном чтении.

Проверены три сценария: файл корректен, файл отсутствует (6a) и в файл добавлено значение «abc» (6b).

```
# Задание 6
# файл main.py
from pathlib import Path

from utils.reader import read_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)
    print("Текущий каталог:", Path.cwd())
    print("Файл существует:", path.exists())
    try:
        numbers = read_numbers(path)
    except FileNotFoundError:
        print("Файл не найден:", path)
    except ValueError:
        print("Файл содержит некорректное значение")
    else:
        print("Значений:", len(numbers), "Мин:",
find_min(numbers), "Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))


if __name__ == "__main__":
    main()
```

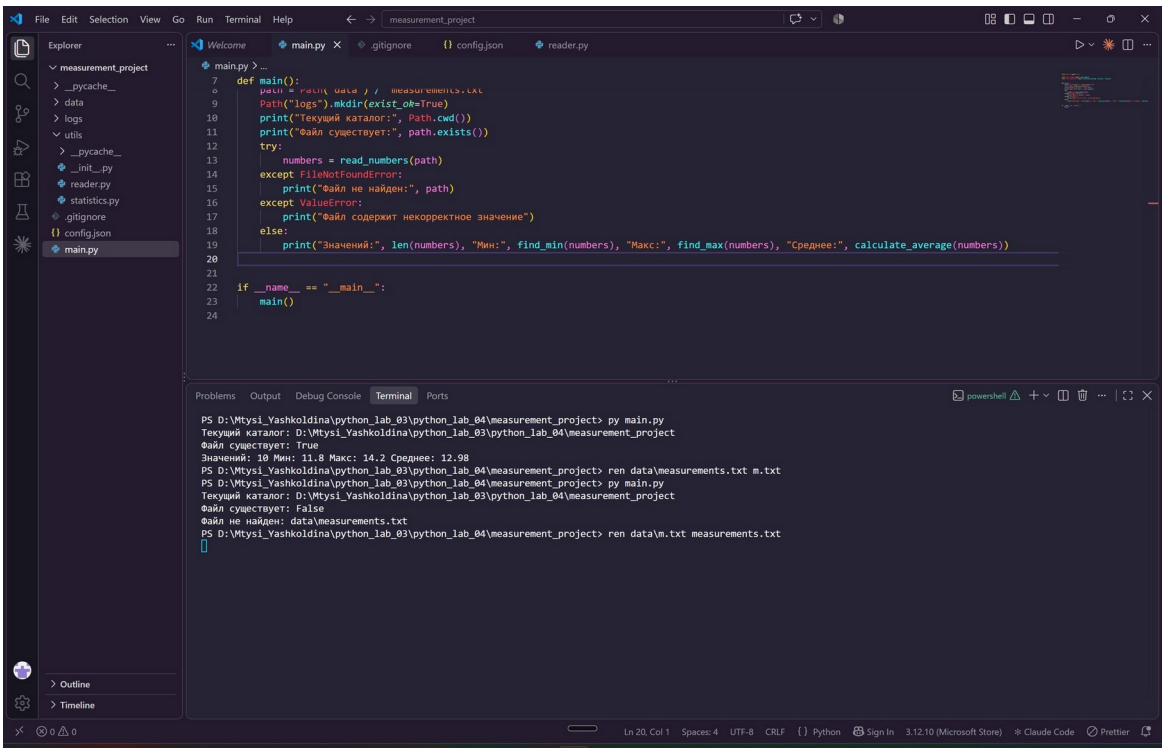


Рисунок 7. – Результат выполнения задания 6 (файл корректен, файл отсутствует).

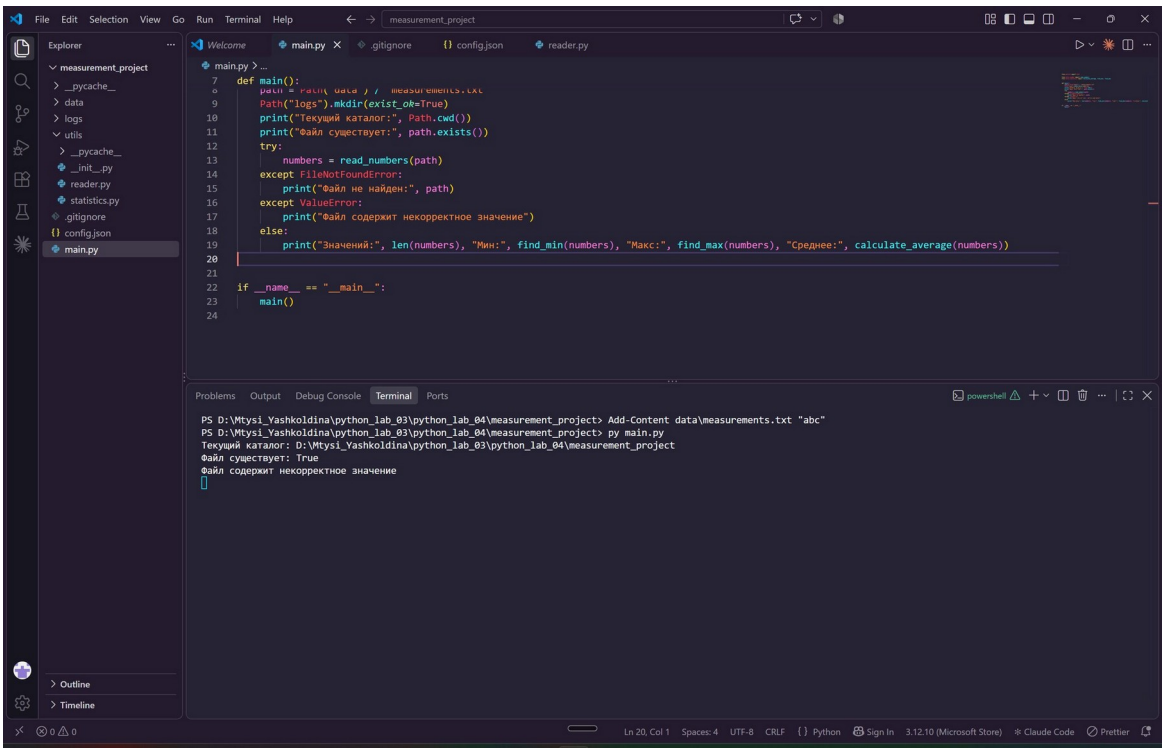


Рисунок 8. – Результат выполнения задания 6 (некорректное значение).

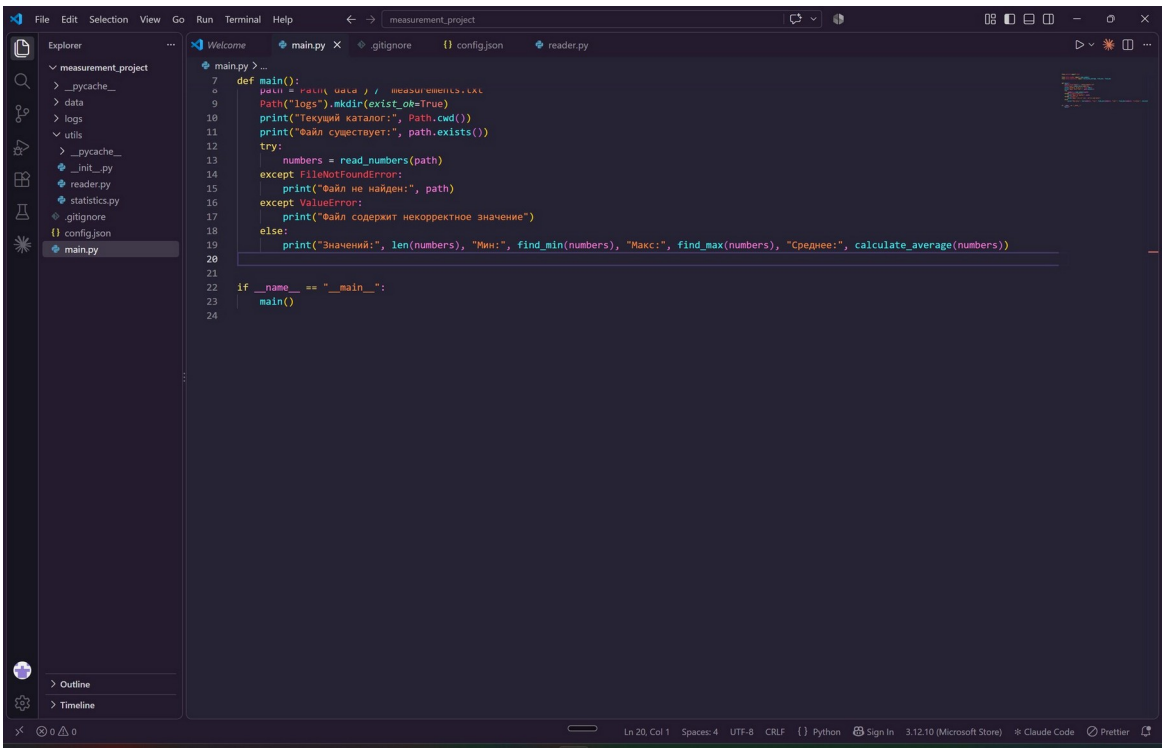


Рисунок 9. – Код main.py задания 6.

7. Проверка данных (raise)

В reader.py добавлена функция validate_numbers(numbers), которая вызывает raise ValueError("Список измерений пуст"), если данных нет. Ошибка перехватывается в main() в блоке except ValueError. Проверка выполнена на пустом файле.

```
# Задание 7
# файл utils/reader.py (добавлена validate_numbers)
def read_numbers(file_path):
    numbers = []
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            line = line.strip()
            if line:
                numbers.append(float(line))
    return numbers


def validate_numbers(numbers):
    if len(numbers) == 0:
        raise ValueError("Список измерений пуст")

# файл main.py
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)
    print("Текущий каталог:", Path.cwd())
    print("Файл существует:", path.exists())
    try:
        numbers = read_numbers(path)
        validate_numbers(numbers)
    except FileNotFoundError:
        print("Файл не найден:", path)
    except ValueError as e:
        print("Ошибка данных:", e)
    else:
        print("Значений:", len(numbers), "Мин:",
find_min(numbers), "Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))


if __name__ == "__main__":
    main()
```

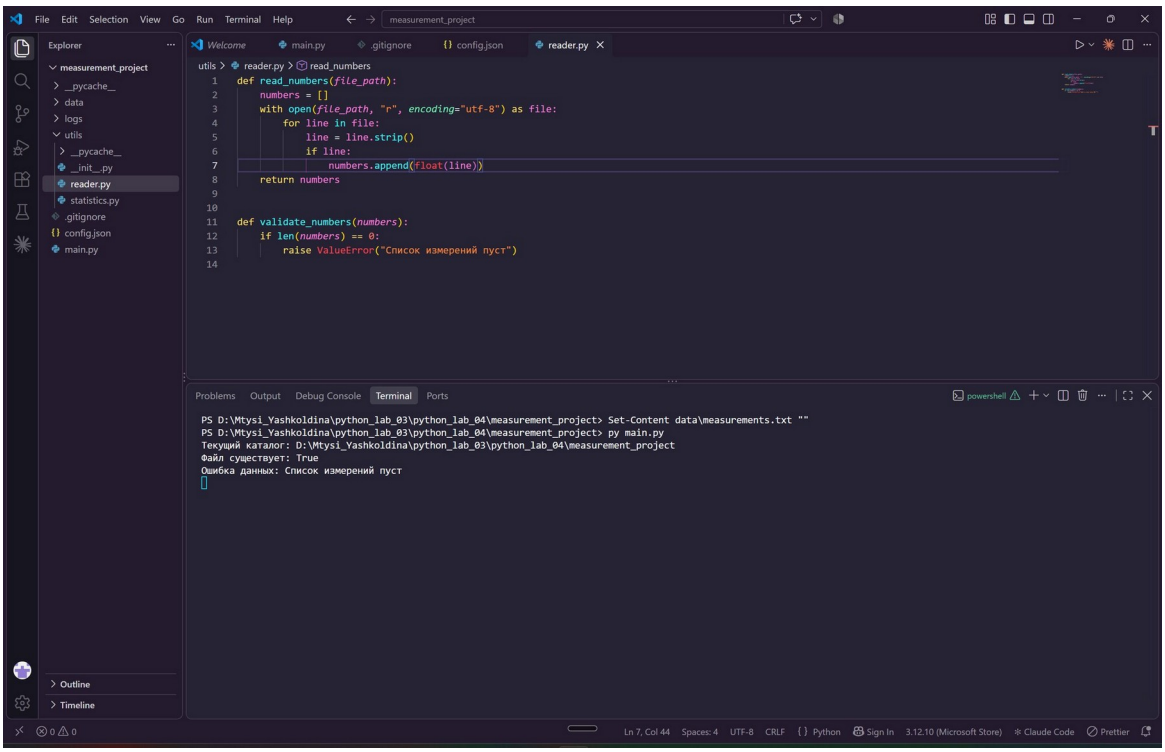


Рисунок 10. – Результат выполнения задания 7.

8. Логирование

Подключён модуль logging: сообщения записываются в файл logs/app.log с временем, уровнем и текстом. Фиксируются запуск программы, успешное чтение, число значений, ошибки (logging.error) и завершение работы (блок finally). На рисунках показаны запуск при корректных данных и при отсутствующем файле (8a) и содержимое журнала (8b).

```
# Задание 8
# файл main.py
import logging
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    path = Path("data") / "measurements.txt"
    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(filename="logs/app.log",
level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s", encoding="utf-8")
    logging.info("Программа запущена")
    print("Текущий каталог:", Path.cwd())
    print("Файл существует:", path.exists())
    try:
        numbers = read_numbers(path)
        validate_numbers(numbers)
    except FileNotFoundError:
        print("Файл не найден:", path)
        logging.error("Файл не найден: %s", path)
    except ValueError as e:
        print("Ошибка данных:", e)
        logging.error("Ошибка данных: %s", e)
    else:
        logging.info("Файл %s прочитан", path)
        logging.info("Прочитано значений: %d", len(numbers))
        print("Значений:", len(numbers), "Мин:",
find_min(numbers), "Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))
    finally:
        logging.info("Программа завершена")


if __name__ == "__main__":
    main()
```

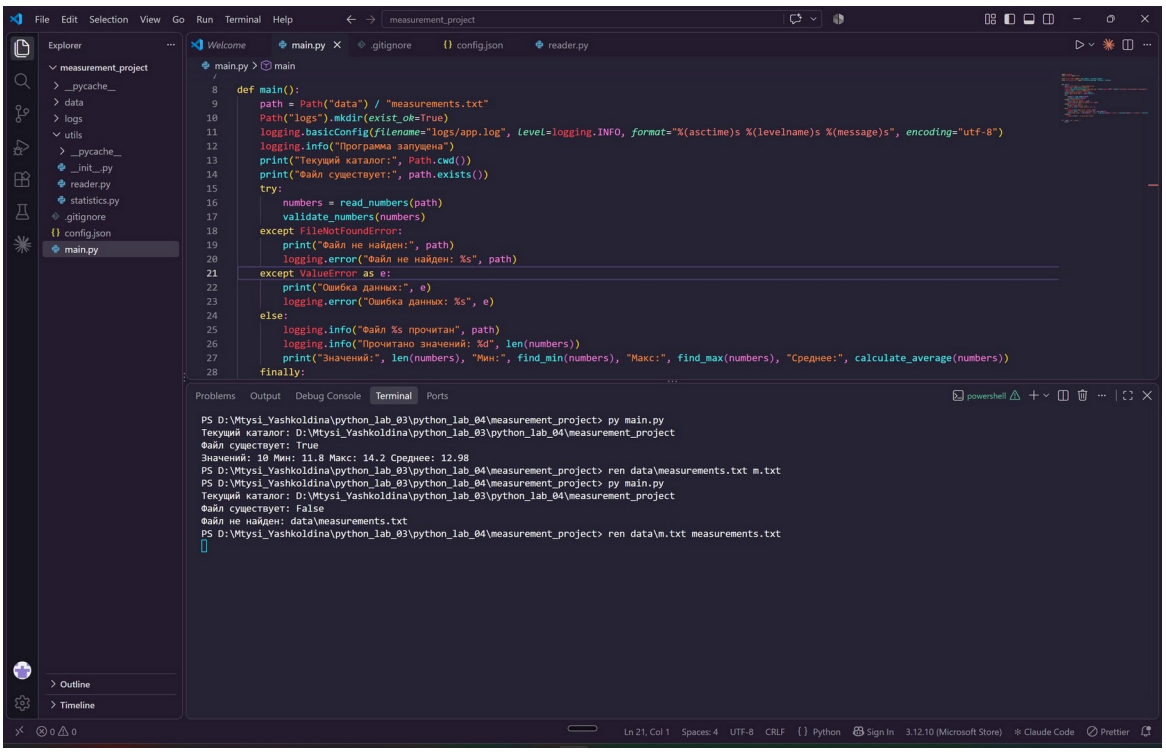


Рисунок 11. – Результат выполнения задания 8 (запуск программы).

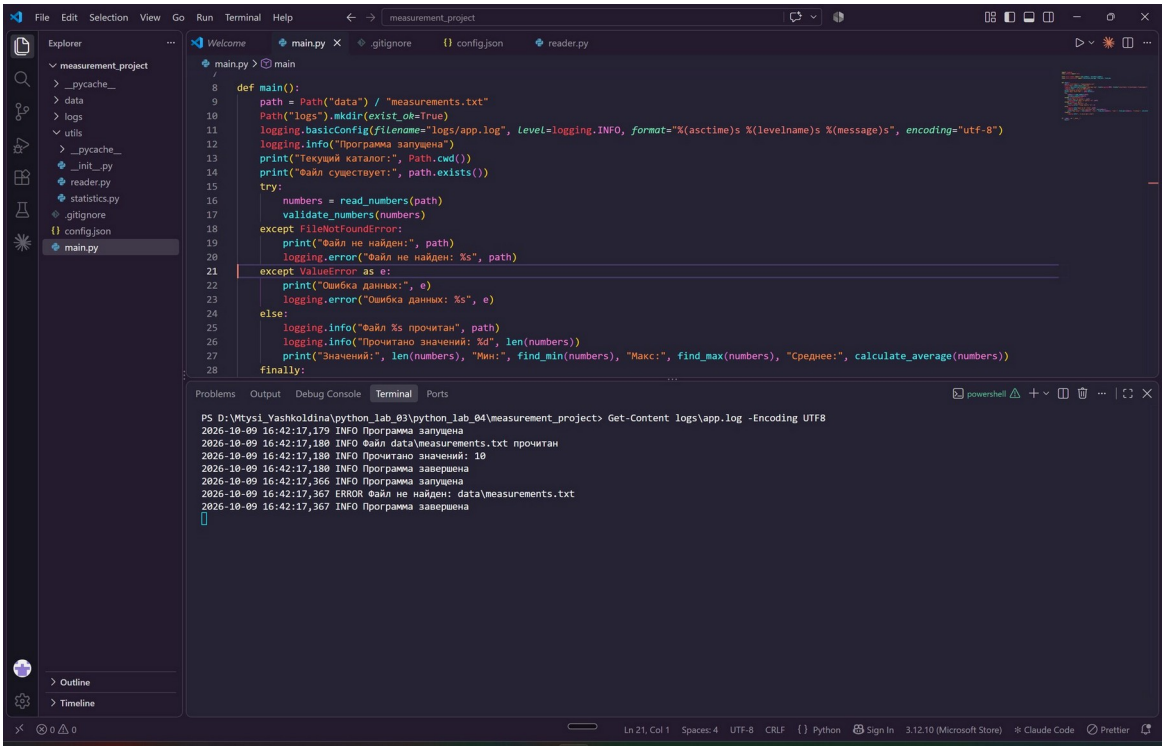


Рисунок 12. – Содержимое журнала logs/app.log.

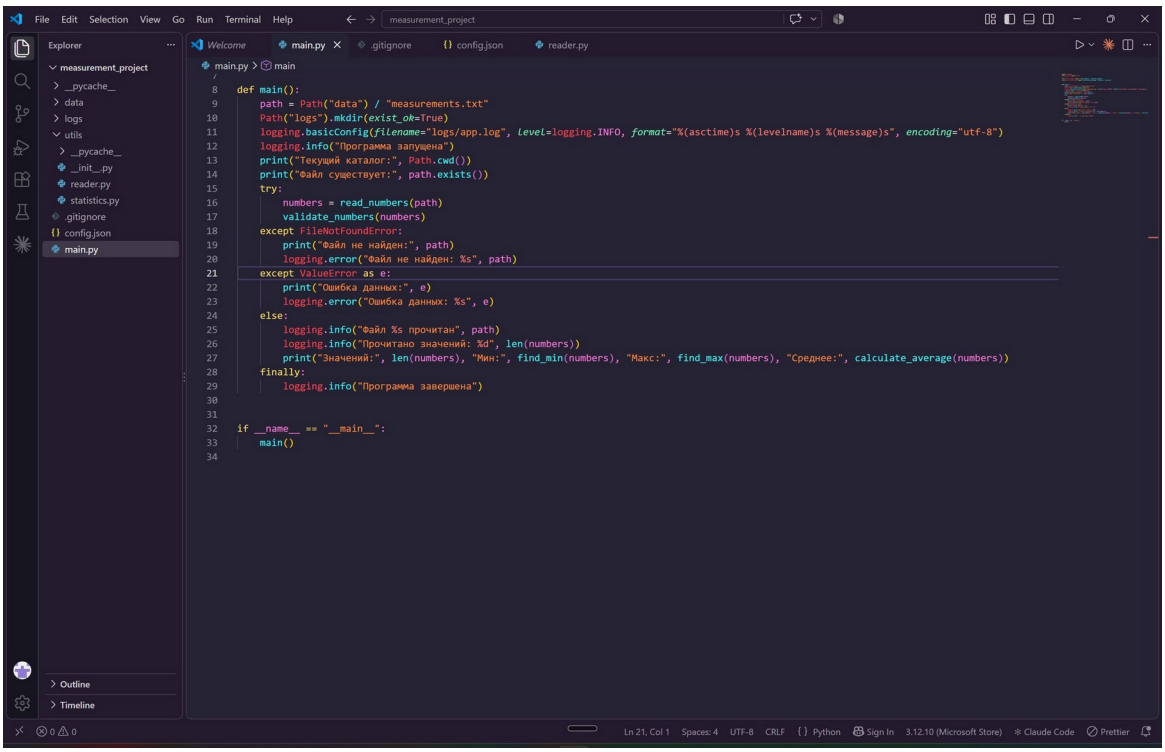


Рисунок 13. – Код main.py задания 8.

9. Настройки из config.json

Пути к входному файлу и журналу вынесены в config.json и читаются модулем json. Для проверки в конфигурации указан другой файл data/other.txt — программа прочитала его без изменения кода, что подтверждает работу настроек.

```
# Задание 9
# файл config.json
{
    "input_file": "data/measurements.txt",
    "log_file": "logs/app.log"
}

# файл main.py
import json
import logging
from pathlib import Path

from utils.reader import read_numbers, validate_numbers
from utils.statistics import calculate_average, find_min,
find_max


def main():
    with open("config.json", "r", encoding="utf-8") as file:
        config = json.load(file)
    path = Path(config["input_file"])
    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(filename=config["log_file"],
level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s", encoding="utf-8")
    logging.info("Программа запущена")
    print("Текущий каталог:", Path.cwd())
    print("Файл существует:", path.exists())
    try:
        numbers = read_numbers(path)
        validate_numbers(numbers)
    except FileNotFoundError:
        print("Файл не найден:", path)
        logging.error("Файл не найден: %s", path)
    except ValueError as e:
        print("Ошибка данных:", e)
        logging.error("Ошибка данных: %s", e)
    else:
        logging.info("Файл %s прочитан", path)
        logging.info("Прочитано значений: %d", len(numbers))
        print("Значений:", len(numbers), "Мин:",
find_min(numbers), "Макс:", find_max(numbers), "Среднее:",
calculate_average(numbers))
    finally:
        logging.info("Программа завершена")


if __name__ == "__main__":
    main()
```

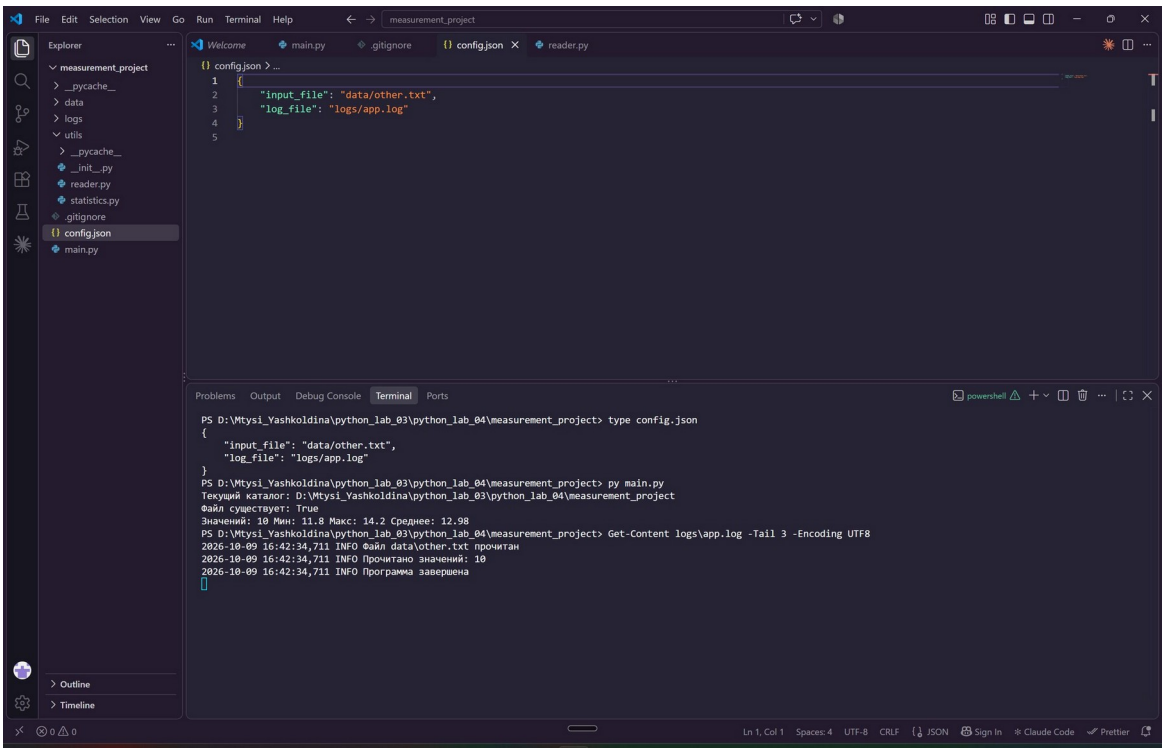


Рисунок 14. – Результат выполнения задания 9.

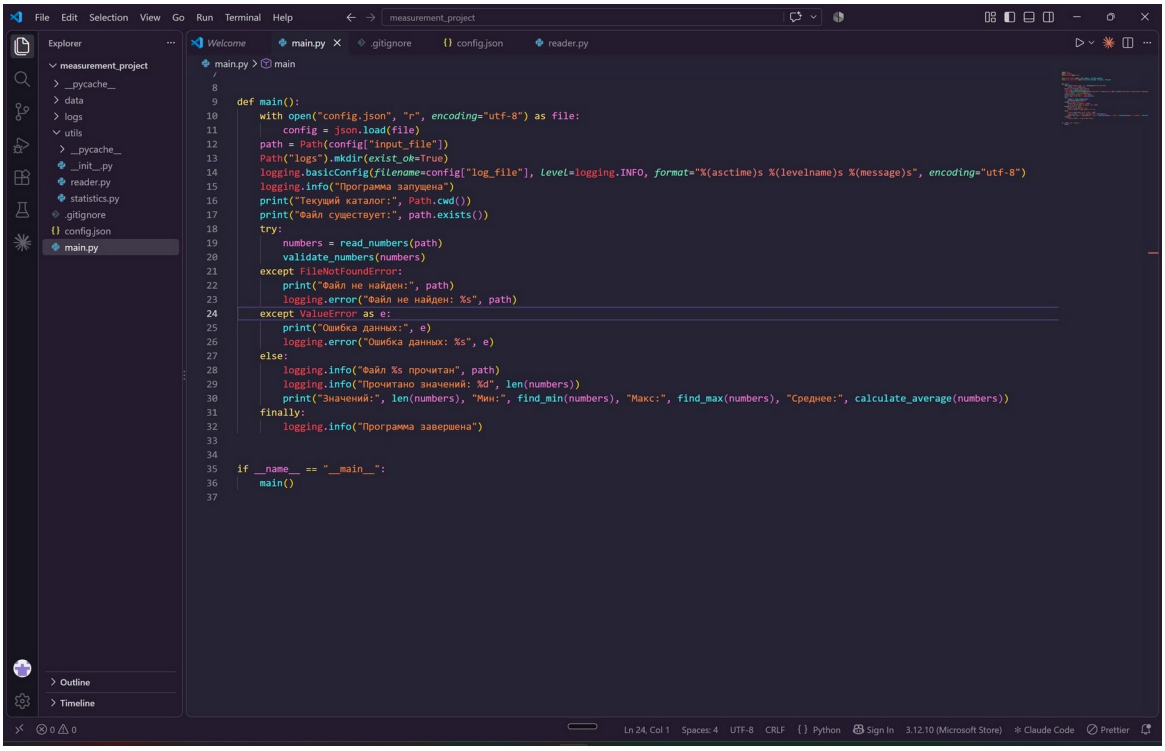


Рисунок 15. – Код main.py задания 9.

10. Виртуальное окружение

Создано виртуальное окружение командой py -m venv .venv и активировано. Команды python --version и where.exe python показывают, что используется интерпретатор из каталога .venv.

```
# Задание 10
# команды PowerShell
py -m venv .venv
.venv\Scripts\activate
python --version
where.exe python
```

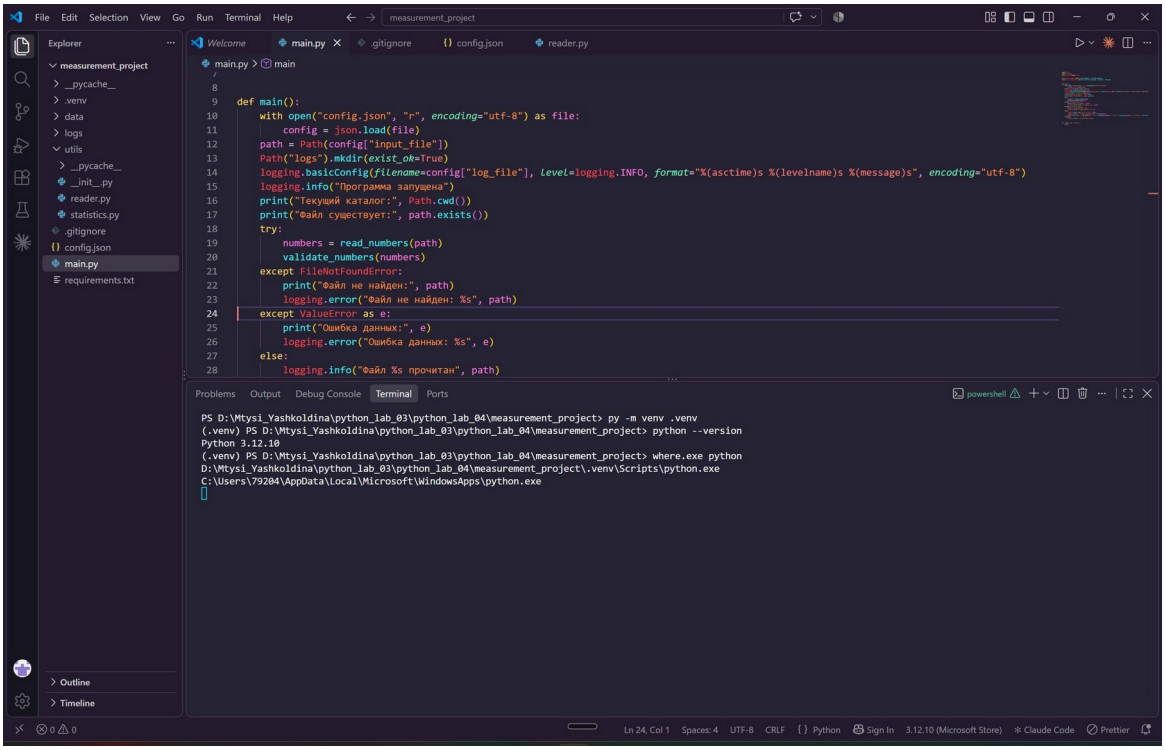


Рисунок 16. – Результат выполнения задания 10.

11. Зависимости (requirements.txt)

Выведен список установленных пакетов (python -m pip list) и сформирован файл requirements.txt командой pip freeze. Он пустой, так как проект использует только стандартную библиотеку и сторонние пакеты не устанавливались.

```
# Задание 11
# команды PowerShell
python -m pip list
python -m pip freeze > requirements.txt
type requirements.txt

# файл requirements.txt
(пустой: используется только стандартная библиотека)
```

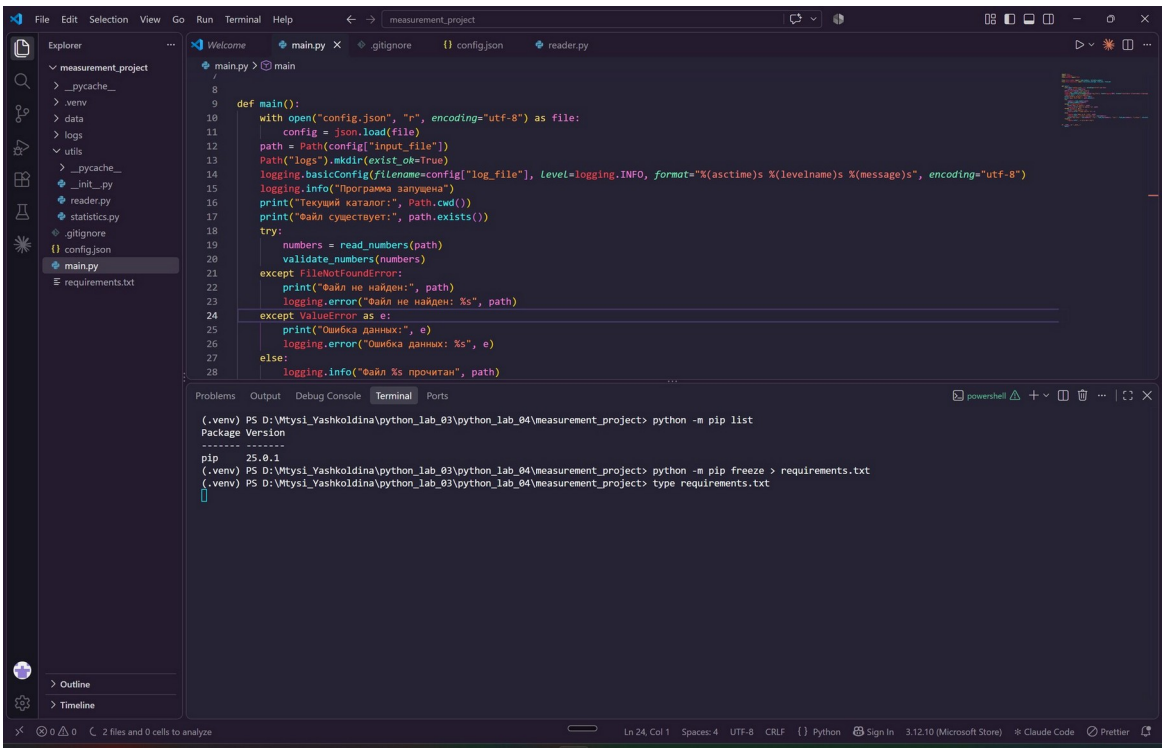


Рисунок 17. – Результат выполнения задания 11.

12. Файл .gitignore

Создан .gitignore, исключающий из репозитория окружение .venv/, кэш __pycache__/, файлы *.pyc и журналы *.log. Командами git status и git check-ignore -v подтверждено, что .venv и logs/app.log игнорируются.

```
# Задание 12
# файл .gitignore
.venv/
__pycache__/
*.pyc
*.log

# команды PowerShell
git status
git check-ignore -v .venv logs/app.log
```

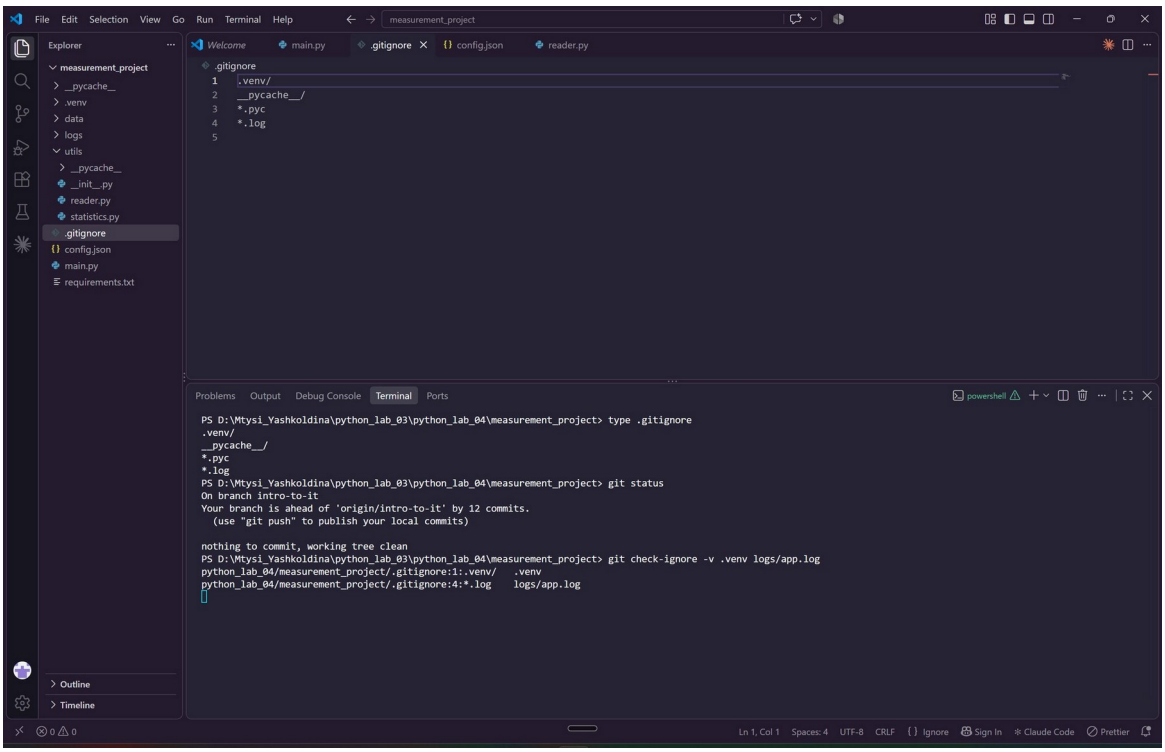


Рисунок 18. – Результат выполнения задания 12.

13. Итоговый проект

Выполнена итоговая проверка проекта. Добавлен пустой файл logs/.gitkeep, чтобы Git сохранил каталог logs. Проверены сценарии: корректные данные (13a), отсутствие файла (13b), некорректное значение (13c); журнал работы показан на рисунке 13d. История изменений оформлена отдельными коммитами по каждому заданию.

```
# Задание 13
# файл logs/.gitkeep
(пустой файл, чтобы Git сохранил каталог logs)

# команды PowerShell
py main.py
ren data\measurements.txt m.txt
py main.py
ren data\m.txt measurements.txt
Add-Content data\measurements.txt "abc"
py main.py
Get-Content logs\app.log -Encoding UTF8
```

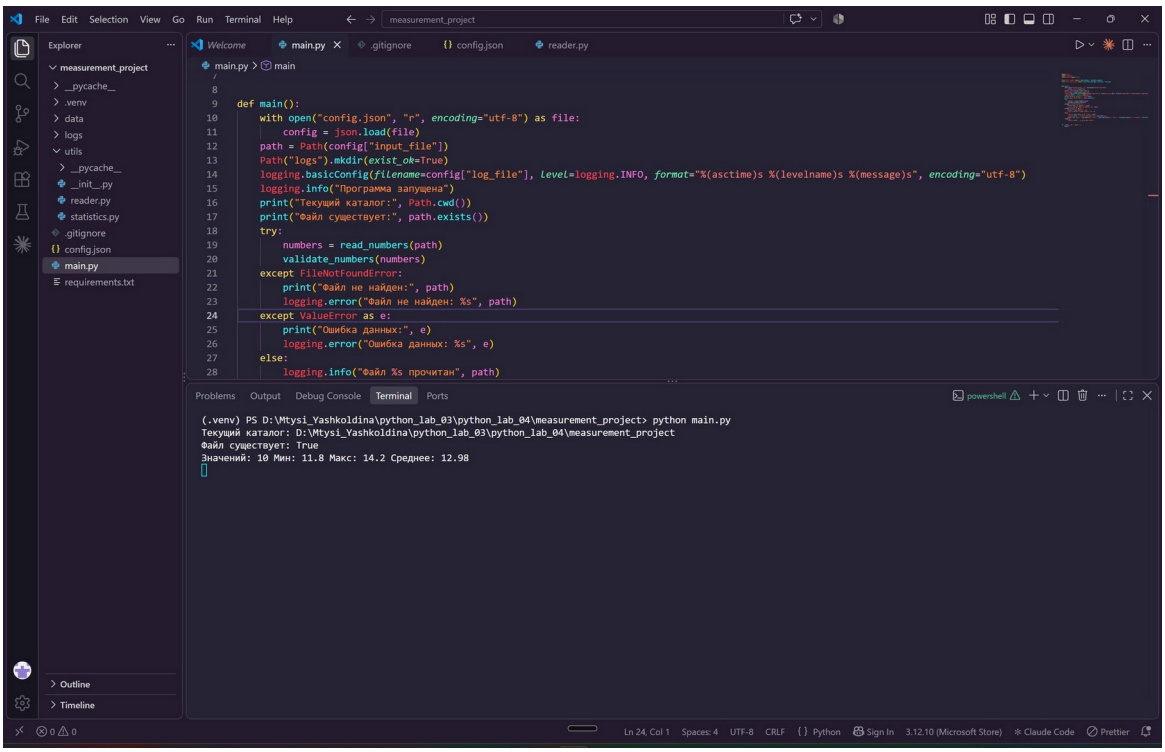


Рисунок 19. – Результат выполнения задания 13 (корректные данные).

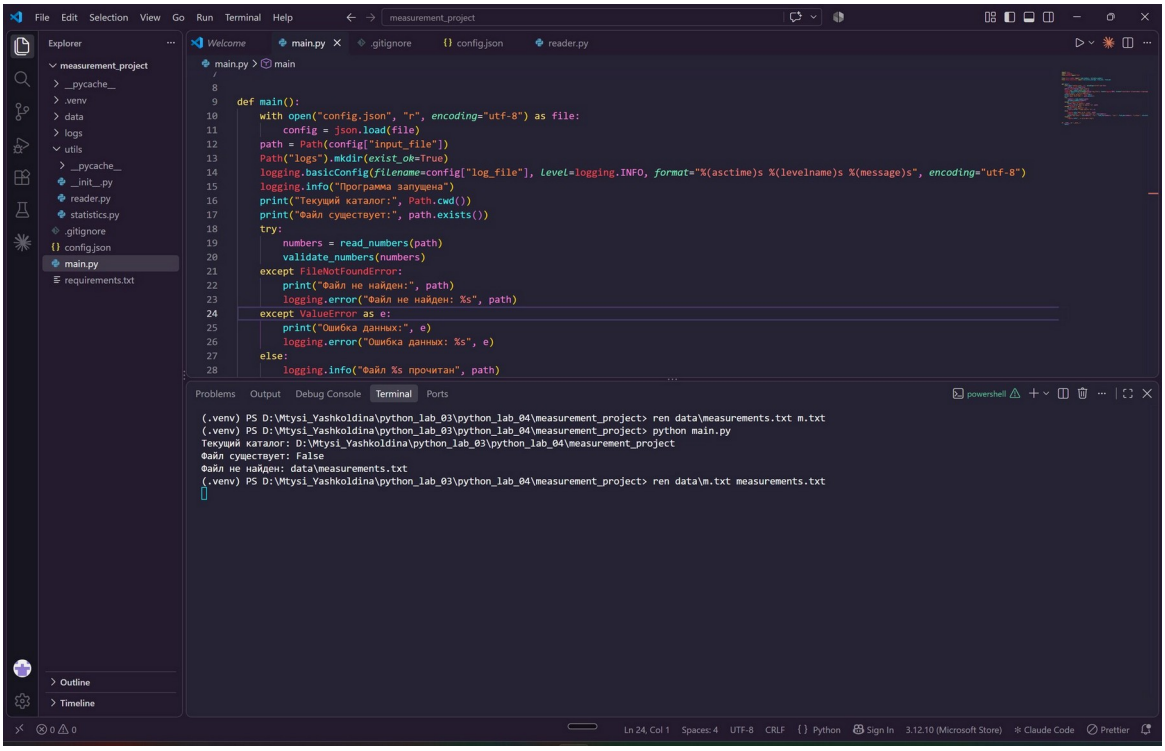


Рисунок 20. – Результат выполнения задания 13 (файл отсутствует).

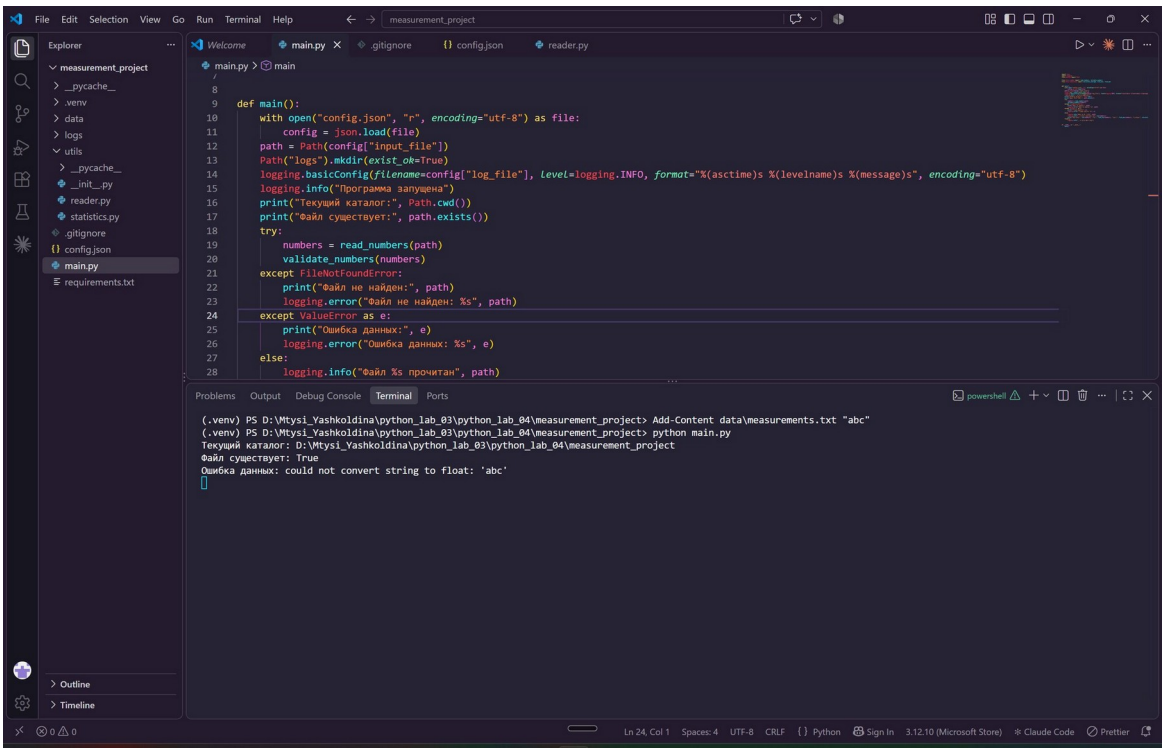


Рисунок 21. – Результат выполнения задания 13 (некорректное значение).

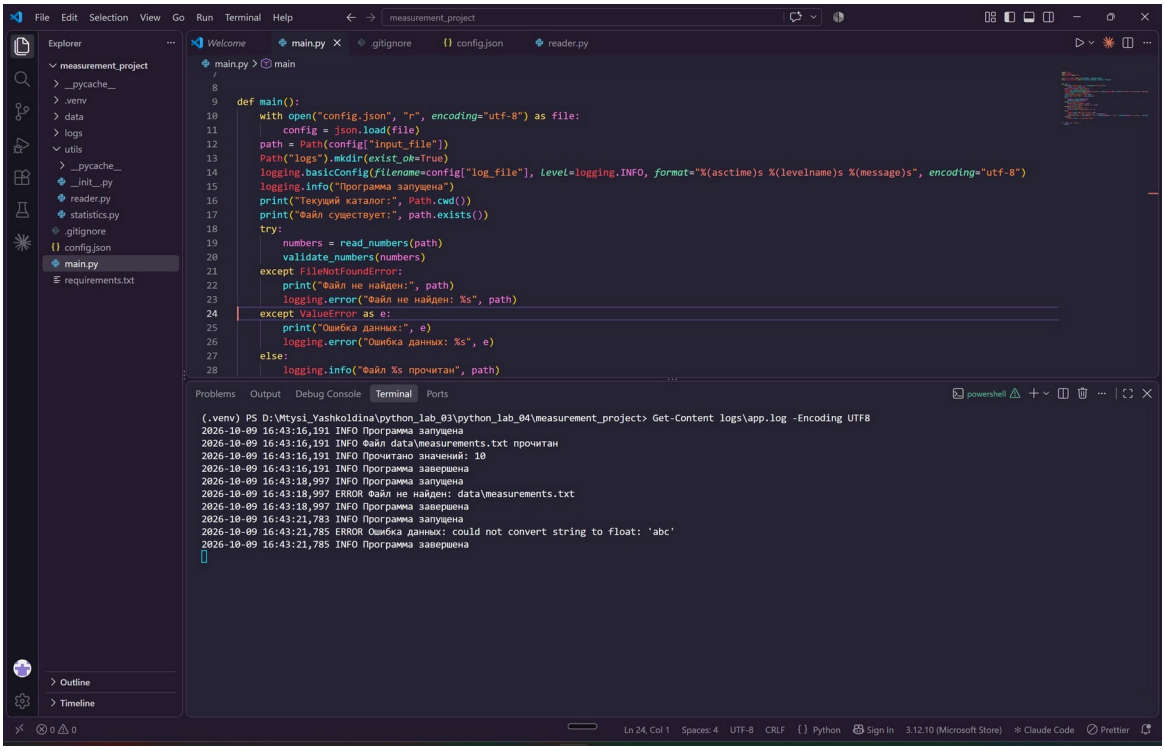


Рисунок 22. – Содержимое журнала logs/app.log.

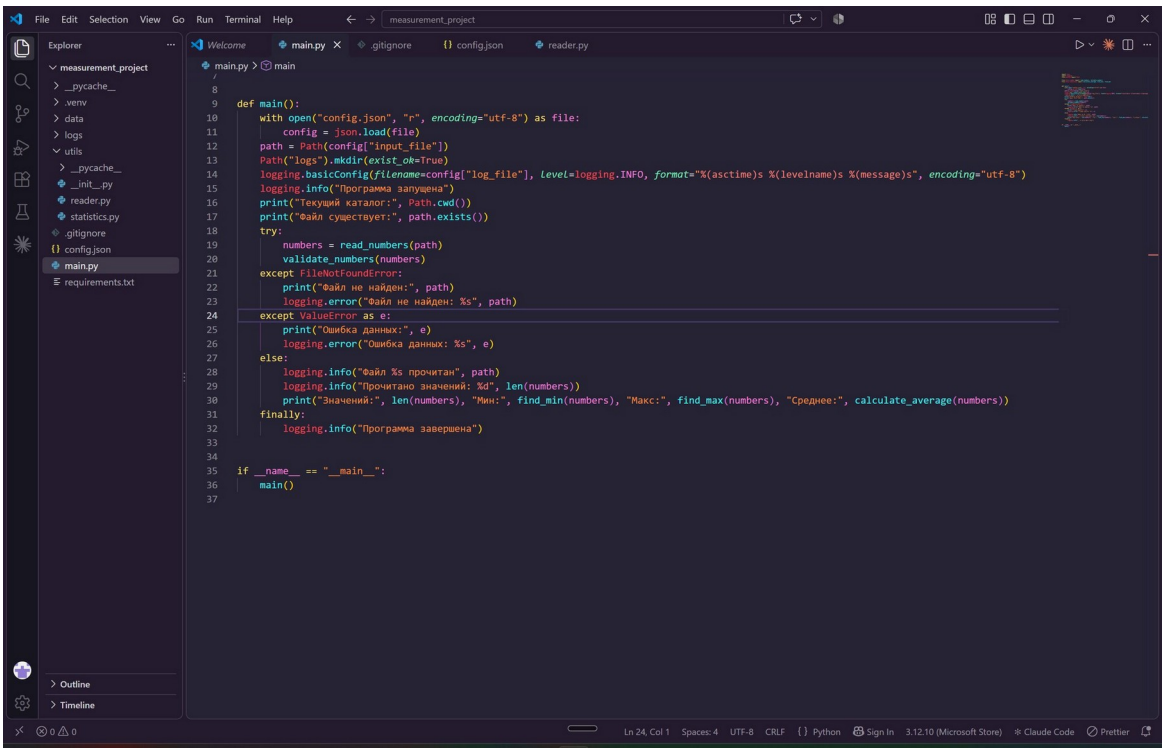


Рисунок 23. – Итоговый код main.py.

Вывод

В ходе лабораторной работы мы получили практические навыки организации Python-проекта: разделили программу на модули и пакет utils, оформили точку входа через if __name__ == "__main__", научились работать с путями через pathlib, обрабатывать ошибки и вызывать исключения, вести журнал работы через logging и хранить настройки в config.json. Также освоили создание виртуального окружения, формирование requirements.txt, настройку .gitignore и ведение истории проекта в Git.